Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Iterative Covering Array Feature Selection for Classification Problems on Waveband Hyperspaces

SALVADOR ROMO[1], MIGUEL DE-LA-TORRE[1,2], (Senior Member, IEEE), HIMER AVILA-GEORGE[2], (Senior Member, IEEE), JOSÉ TORRES-JIMENEZ[3], (Senior Member, IEEE), AND WILSON CASTRO.[4], (Senior Member, IEEE)

[1]Instituto Tecnológico de Estudios Superiores de Occidente, Boulder, CO 80305 USA (e-mail: salvador.romo.macias@gmail.com)
[2]Universidad de Guadalajara, Depto. de Ciencias Computacionales e Ingenierías, Ameca, Jalisco, México
(e-mail: miguel.dgomora, himer.avila@academicos.udg.mx)
[3]UNIDAD TAMAULIPAS, CINVESTAV, Parque Científico y Tecnológico TECNOTAM – Km. 5.5 carretera Cd. Victoria-Soto La Marina C.P., 87130 Cd, Victoria, Tamps, Mexico (e-mail: jtj@cinvestav.mx)
[4]Universidad Nacional de Frontera, Sullana, Piura, Peru (e-mail: wcastro@unf.edu.pe)

Corresponding author: Miguel De-la-Torre (e-mail: miguel.dgomora@academicos.udg.mx).

The PROCIENCIA program supported this work found through Grant PE501078266-2022 'Redes convolucionales y combinaciones de bandas espectrales e índices vegetativos para identificación de árboles plus de algarrobo (Prosopis pallida): aplicación de técnicas Deep learning e imágenes multiespectrales', and 'Creation of the Food Safety Research Laboratory Service at the National University of the Frontier" with code No. 2439545.

ABSTRACT Searching for the most relevant features for classification or regression, also known as feature selection (FS), comprises a set of techniques that are widely employed to reduce and improve the performance of machine learning models. In that sense, covering arrays (CA) are combinatorial data structures used in experiment design that have been used for efficient FS in chemometrics. The covering array feature selection (CAFS) algorithm for supervised FS was previously proposed to reduce the amount of test cases in the process of finding the most relevant features for classification. However, the CAFS algorithm reduces the feature space by halving a same CA, assuming the dependency among features is preserved by the same structure. In this paper, an extension of the CAFS algorithm is proposed to iteratively generate a new CA after each reduction step to avoid such an assumption. The new algorithm is called iterative CAFS (ICAFS), and employs the IPOG algorithm to generate a new *ad-hoc* covering array for every new subset of features. The algorithm was tested using three cases of study: an expanded circle in square synthetic data, the 6-class hyperspectral cacao dataset, and the multispectral algarrobo dataset. Results reveal that the ICAFS algorithm outperforms the previous CAFS approach in all test datasets.

INDEX TERMS Covering Arrays, Feature Selection, Classification, Algarrobo, Remote Sensing

I. INTRODUCTION

FEATURE selection methods involve a set of techniques employed to reduce high-dimensional data sets into smaller and potentially more accurate subsets of features. In the best cases, the objective is to remove irrelevant, redundant and noisy features that negatively affect machine learning (ML) algorithms, either for classification, regression or prediction. By removing these less relevant or noisy features, the dataset is potentially clean of non-relevant information and is more representative of the problem to be modeled through machine learning algorithms. Those features that imply the co-precense of others have a high correlation. In other words, a feature alone is relevant for the task, but removing the correlated ones will not affect negatively the learning performance. On the other hand, other features may not be relevant for classification, either if they are independent of the rest. Even more, a noisy and non-relevant feature alone can represent a decrease in classification accuracy, but two noisy features can complement each other to distinguish samples from different classes. So, by applying feature selection algorithms, it overcomes the following problems **1.** models that gets overfitted **2.** degeneration of algorithms performance due to highly dimensional data sets and **3.** usage of a lot of computational resources Guyon and Elisseeff

[2003], Wang et al. [2016].

The importance of feature selection on ML applications is hard to underestimate, given the possibility to reduce computational complexity, with the associated cost. As an example in chemometrics, Castro et al. [2022] came across with a highly dimensional problem to classify cacao nibs in six different cacao varieties. The quality of the cacao derived products is specially relevant for food industry, and represents a trade-off between cost and quality. The dataset employed by Castro et al. [2022] presented the 1401 values for each measurement, generated using *near-infrared spectroscopy* techniques. The resulting data represents a high dimensional vector for each sample, and both measurement and classification require both highly specialized sensors, and high computational equipment. So, to tackle this problem and reduce the dimensionality of the cacao dataset, they propose the *CAFS (Covering Array Feature Selection)* algorithm, that is capable to reduce from 1401 features to only five, preserving a high accuracy.

The motivation of this paper is not only explore in depth the algorithm presented by Castro et al. [2022] and compare it against three different well-known nature-inspired algorithms available on the open source Python library called *Py FS* implemented by Guha et al. [2021]; but also to give researchers and practitioners a broad analysis of FS algorithms.

Structure of the paper. In **Section 2** the different categories of feature selection approaches are exposed, as well as a brief explanation of the feature selection used for experiments. **Section 3** provides the details of the methodology and the metrics selected to measure the performance of the different algorithms addressed in the experiment. **Section 4** presents the results and discussion retrieved after experiments. Finally, **Section 5** presents the conclusion and future work.

II. FEATURE SELECTION ALGORITHMS

According to Wang et al. [2016], feature selection algorithms can be categorized in six main groups, considering their underlying characteristics:

1) **Supervised Feature Selection**: These algorithms work better on data that is fully labeled because it can select discriminative features to distinguish sample and instead of using all the data to train the model, it first selects a subset of features and then processes the data with the selected features to the learning model. The feature selection phase will use the label information as well as the information gain from each feature or Gini index.
2) **Unsupervised Feature Selection**: These type of algorithm assumes that no label is present on the data to be processed, hence, it has to relay on two main approaches; first approach is to seek cluster indicators and simultaneously apply supervised feature selection and second is to sequentially seek cluster indicators and apply supervised feature selection selection iteratively.
3) **Semi supervised Feature Selection**: One common characteristic that these datasets presents is that the labels are partially presents and the labeled data that is actual present is insufficient to represent data distribution. So before taking a step and select features with supervised approaches, these approach construct a similarity matrix and select features that best fit the matrix.
4) **Filter Methods**: its fully based on the characteristics without using learning algorithms. Since these algorithms doesn't consider the bias or heuristic of learning algorithms, it may missed relevant features. the template of this algorithm consist on two steps, first step is to rank features on certain criterion; second step is to select features with the highest rank. The criterion selection consist of different statistics techniques applied around the features.
5) **Wrapper Methods**: These methods focuses on using learning algorithms as bias or heuristics to guide the search on finding the subset with the highest accuracy. The only draw back of this kind of methods is that it has look over $O(2^m)$ combinations on the feature space. That's why is usually combined with different search strategies as genetic algorithms, hill-climbing, meta-heuristic algorithms and more.
6) **Embedded Methods**: The big difference among filter and wrapper method is that the feature selection is performed internally at the construction of the model, taking advantages of less computationally intensive and interaction with the learning model.

After exploring the different definition for each Feature Selection method, [Guyon and Elisseeff, 2003, Liu and Motoda, 2007] addressed the above mentioned definition in a similar way, and they explain in detail different techniques, algorithms and related tools on the different categories previously discussed.

The focus of this experiment goes over the wrapper method, because most of the algorithms found on PyFS lays under this category. Guha et al. [2021] created a reliable open source python library that implements a total of 12 meta-heuristic based algorithms and only a few for Filter-based method, such algorithms are listed on the next table. On the other hand the algorithm proposed by Castro et al. [2022] can also be considered under this category.

This paper will use one solid implementation of a Binary Bat Algorithm available on PyFS and discovered by Nakamura et al. [2013]; such algorithm will be compared against two covering array based algorithm; the first is *CAFS*, while the second is a new variation of the *CAFS* with some important improvements presented on this paper, named as *CAFSI (Covering Array Feature Selection Iterative)* . Before jumping and explore the methodology to be used on the experiment, it is important to briefly explain key concepts used on the algorithms to use.

A. COVERING ARRAYS

A covering array is a combinatorial data structure defined by the function $CA(N, k, t, v)$, where N and k stands for the number of rows and columns. t for the strengths or interaction between the columns and v is the size of alphabet defined by $\mathbb{Z}_v$.

Let's see an example for the next Covering Array (point to image and create a covering Array) $CA(12, 7, 2, 3))$. The alphabet is $Z_v = 0, 1, 2$ because each cell of the matrix can take three values of the set $\{0, 1, 2\}$, here $N = 12$ and each row represents a different test case for the system, and $k = 7$ are the different parameters to test in the system. Finally, t represents the interaction between two parameters such that each of the $\binom{7}{2}$ sub-arrays formed by $t = 2$ columns has as a row v^t tuples that extend over $\mathbb{Z}_v$; this means, for the matrix (point to the matrix) to meet the condition to be a covering array, each of the $3^2 = 9$ sub-arrays composed of 2 columns has to cover these $v^t = [(0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2)]$ tuples.

Many of the problems around *CA* focus on finding the least number of test cases or rows to cover all interactions between t parameters, such problems can be defined with the following function $CAN(k, t, v)$. Izquierdo-Marquez et al. [2018] explores a complex 3-stage algorithm combining meta-heuristics and greedy approaches based on $CPHF$ or *Covering Perfect Hash Families* insertar bibliografia de CPHF). Torres-Jimenez et al. [2019] compiles different covering array construction algorithms divided into two main categories, Combinatorial and searching methods.

Finally (encontrar referencia del parámetro t deseado). Among the two *CA* Feature Selection algorithms presented in this paper, only *ICAFS* will use a greedy algorithm for the construction of *CA* , because on each iteration it will need to construct a new *CA*, given a new set of selected features. On the other hand, *CAFS* needs to work under a CA previously constructed.

TABLE 1: $CA(12, 7, 2, 3)$ with $t = 2$, $Z_v = \{0, 1, 2\}$,$k = 7$ and $N = 12$ because every sub array of length 2 covers at least once all possible tuples formed from $Z_v = \{0, 1, 2\}$

$$\begin{bmatrix} 1 & 2 & 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 2 & 2 & 2 & 1 & 2 & 1 & 2 \\ 2 & 1 & 0 & 1 & 0 & 2 & 1 \\ 0 & 0 & 2 & 1 & 1 & 0 & 1 \\ 0 & 2 & 0 & 2 & 1 & 2 & 2 \\ 2 & 2 & 1 & 2 & 0 & 0 & 0 \\ 1 & 1 & 2 & 0 & 0 & 0 & 2 \\ 1 & 0 & 2 & 2 & 2 & 2 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 2 & 0 & 1 & 0 & 1 & 2 & 2 \\ 0 & 1 & 1 & 2 & 2 & 1 & 1 \end{bmatrix}$$

B. BINARY BAT ALGORITHM

Bat algorithm is a meta-heuristic bio-inspired technique discovered by Yang [2011] and inspired by the behaviour of the bats on how a swarm of micro bats locate their prey/food using echolocation. Yang [2011] defined the behavior of micro-bats as follows

- All bats use echolocation to sense distance, and they know the difference between food/prey
- A bat b_i flies randomly with velocity v_i at position x_i with a fixed frequency f_{min} , varying wavelength λ, and loudness A_0 to search for a prey. They can adjust the wavelength of their emitted pulses and their rate of pulse emission on behalf of the proximity of a target.

These behaviours are translated on the equations 1,2 and 3

$$f_i = f_{min} + (f_{max} - f_{min})\beta \tag{1}$$

$$v_j^i(t) = v_i^j(t-1) + [\hat{x}^j - \hat{x}_i^j(t-1)]f_i \tag{2}$$

$$x_i^j(t) = x_i^j(t-1) + v_j^i(t) \tag{3}$$

This algorithm belongs to multi-objective optimization category. This means that in order to get the best possible solution for a problem, the algorithm needs to optimize a set of values $M_i|i = 0, 1, 2, \ldots, N$; so, one of the most important elements in the design of each micro-bat is a vector $\vec{v}$ that will hold each value to be optimized.

The eq.1 represents the frequency on which each micro-bat will emit echolocation, β is a randomly generated number between the range $[0, 1]$; then, the result from eq.1 controls the pace and range of the movement of the bats and the variation in the wavelengths' frequency that each bat is going to have at each iteration. $x_i^j(t)$ represents the best partial solution for microbat x_i in the value j_{th}. $\hat{x}^j$ is the maximum solution globally found so far among the swarm of microbats. Eq.2 adjusts the velocity of each microbat to reduce the distance toward the best possible solution found on each v_i for each microbat. Eq.3 just updates the new position with the velocity calculated on eq.3. Yang [2011] came up with an additional step to improve the variability on each micro-bat x_i^j that consisted in including random walks depicted on eq.4.

$$x_{new} = x_{old} + \epsilon \bar{A(t)} \tag{4}$$

where $\bar{A(t)}$ stands for the average loudness of all bats at iteration t and ϵ its a random number between the range $[1, -1]$.

Nakamura et al. [2013] developed the Binary Bat algorithm for feature selection that aims to produce random binary N points result of applying the before mentioned equations; but since the result of each point is not binary, Nakamura et al. [2013] introduced a sigmoid function to convert each point into a binary value.

The meaning of each point for each bat_i into this algorithm means if a feature is selected with the value 1 or 0 otherwise,

and $N = len(labels(dataset))$, equation 5 to 6 shows how each optimized point will be passed to a binary alphabet.

$$S(v_j^i) = \frac{1}{1 + e^{-v_j^i}} \tag{5}$$

$$S(v_j^i) = \begin{cases} 1, & \text{if } S(v_j^i) > \delta. \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

the resulting collection of points selected as 1 or 0 is evaluated against a learning algorithm and the accuracy for the model created from the features selected on each is taken as the fit metric; so the bat with the best accuracy is fit among the T iteration will be the one that contains the best features. The learning algorithm used for this problem is Optimum path tree forest, developed by (cita,). The binary bat algorithm is shown on Algorithm 1

Algorithm 1 Binary Bat Algorithm

for $b_i(\forall i = 0, 1, 2 \ldots, m)$ **do**
  for $j(\forall j = 0, 1, 2 \ldots, m)$ **do**
    $x_i^j \leftarrow Random(0, 1)$
  end for
  $v_j^i(t) \leftarrow 0$
  $A_i \leftarrow [1, 2]$
  $r_0 \leftarrow [0, 1]$
end for
for $t(\forall t = 1, 2, \ldots, T)$ **do**
  for $b_i(\forall i = 0, 1, 2 \ldots, m)$ **do**
    Create Z_1' and Z_2' from Z_1 and Z_1 such that both contain only features on b_i in which $x_i^j \neq 0$, $\forall i = 0, 1, 2, \ldots, n$
    Create model with Z_1' and test over Z_2', then get accuracy and store it on acc
    $rand \leftarrow Random[0, 1]$
    if $rand > A_0$ and $acc > fit_i$ **then**
      $fit_i \leftarrow acc$
      $A_i \leftarrow \alpha A_i$
      $r_i \leftarrow r_i^0[1 - e^{\gamma t}]$
    end if
  end for
  $[maxfit, maxindex] \leftarrow max(fit)$
  if $maxfit > globalfit$ **then**
    for $b_{maxfit}(\forall j = 0, 1, 2, \ldots, n)$ **do**
      $x^j \leftarrow x_{maxfit}^j$
    end for
  end if

From the Algorithm 1 one can see equations 7 to 8 that control the pulse rate and the loudness at each iteration.

$$r_0(t+1) = r_0(t) + [1 - e^{\gamma t}] \tag{7}$$

$$A_i(t+1) = \alpha A_i(t) \tag{8}$$

where α and γ are constants purposely added and initially initialized between the range $[1, 2]$ for loudness and $[0, 1]$ for the pulse ratio.

Algorithm 2 Binary Bat Algorithm PT 2

  for $b_i(\forall i = 0, 1, 2 \ldots, m)$ **do**
    $\beta \leftarrow Random[0, 1]$
    $rand \leftarrow Random[0, 1]$
    if $rand > r_i$ **then**
      for $j(\forall j = 0, 1, 2 \ldots, n)$ **do**
        $x_i^j \leftarrow x_i^j + \epsilon \bar{A}$
        $\sigma \leftarrow Rando[0, 1]$
        if $\sigma < 1(\frac{1}{1+e^{-x_i^j}})$ **then**
          $-x_i^j \leftarrow 0$
        else
          $-x_i^j \leftarrow 1$
        end if
      end for
    end if
    $rand \leftarrow Random[0, 1]$
    if $rand > r_i$ and $fit_i < globalfit$ **then**
      for $j(\forall j = 0, 1, 2 \ldots, n)$ **do**
        $f_i \leftarrow f_{min} + (f_{max} - f_{min})\beta$
        $v_j^i \leftarrow v_i^j + [\hat{x}^j - \hat{x}_i^j]f_i$
        $\sigma \leftarrow Rando[0, 1]$
        if $\sigma < 1(\frac{1}{1+e^{-v_j^i}})$ **then**
          $-x_i^j \leftarrow 0$
        else
          $-x_i^j \leftarrow 1$
        end if
      end for
    end if
  end for
end for
return x^j

C. IPOG ALGORITHM

IPOG its an enhancement of the IPO algorithm introduced by Lei and Tai [1998]. IPO aims to reduce the testing of n inputs on a software system by finding a **pair cover** with the least numbers of tests $\mathcal{T}'$ such that each $\tau \in \mathcal{T}'$ covers at least one (α, β) in $\mathcal{P}$. This algorithm is classified under the greedy methods for generating covering arrays and works by firstly creating a matrix with the permutations of the first n parameters and starts growing horizontally by adding to the existing τ_i, the next $v \in P_i$ and calculating if the current $\mathcal{T} + P_i$ can cover at least once each tuple that can be formed from the a values of the $P_0 \ldots P_i$ parameters. if this was not possible, a second routine called horizontal growth enters to the field to create a new test (row) to cover the missing tuples of every value of every $P_0, P_1, \ldots P_i$ parameters. IPO was great for pair-testing but Lei et al. [2007] presented IPOG to extend the algorithm to create test suites to $t > 2$.

IPOG follows the same approach of IPO, but to be sure that all t tuples of values of n parameters are covered at least once, it introduces an optimized data structure to track whether the existing test τ_i has at least covered a t tuple. Its important

to mention that these two algorithms are deterministic, so every time it will be generating the same covering array. The Algorithm 3 describes the IPOG introduced by Lei et al. [2007].

Algorithm 3 IPOG (t,P)

$ts \leftarrow \{\}$
denote the parameters in P, in an arbitrary order, as $P_1, P_2, \ldots, P_n$
add into test set ts a test for each combination of values of the first t parameters
for $i \leftarrow t+1$ to P_n **do**
    $\pi \leftarrow t$ way combination of values involving the Parameter P_i and $t-1$ parameter among the first $i-1$ parameter
    for $\tau = (v_1, v_2, \ldots, v_{i-1}) \in ts$ **do**
        for $v_i \in P_i$ **do**
            $\tau' = (v_1, v_2, \ldots, v_{i-1}, v_i)$
            if τ' covers the most t tuples from π **then**
                $\tau = \tau'$
            end if
        end for
    end for
    //vertical extension for parameter P_i
    for $\sigma \in \pi$ **do**
        if $\sigma \in ts$ **then**
            remove σ from π
        else
            change an existing test to cover σ if possible, otherwise add a new test to cover σ
        end if
    end for
end for

D. COVERING ARRAY FEATURE SELECTION

CAFS its an algorithm first introduced by Castro et al. [2022] to tackle the problem of high dimensionality on a data set with more than a thousand features. This algorithm was designed around Naive Bayes classification algorithm, but this implementation is done using *kNN* algorithm.

One requirement for this algorithm to work it's a Covering Array with the following properties : $CAN(k, t, v)$. The number of k rows is equal to the number of features and the alphabet is binary because the matrix will consist of 1 and 0. Once the pre-condition are established we can proceed to explain *CAFS*.

This algorithm sets the $maxFeatureProspects$ with the current labels presents on the current $dataSetX$. At the same time, $maxNumberOfFeatures$ is initialized with the count of present features on the training set.

It is important to mention that this counting has to match the number of columns on the covering array. Then, while the breaking condition is met, that in this case its when the max number of iteration is reached or when there is only one feature left on $maxNumberOfFeatures$ and its not possible

Algorithm 4 CAFS(CA, $dataSetX$, $dataSetY$)

if $len(CA) \leq len(TrainingSetX)$ **then**
    $CAFS$ can't be applied with small CA
    return $\{\}$
end if
$globalMaxScore \leftarrow 0.0$
$numRows \leftarrow CA.rows$
$numCols \leftarrow CA.cols$
$maxFeatureProspects \leftarrow labels(dataSetX)$
$maxNumberOfFeatures \leftarrow count(dataSetX)$
while $i \leq maxIterations$ **do**
    $currMaxFeatureProspects \leftarrow \{\}$
    $currMaxScore \leftarrow 0$
    for i in $range(0, numRows)$ **do**
        $auxMaxFeatureProspects \leftarrow \{\}$
        for j in $range(0, numCols)$ **do**
            if $CA[i][j] = 1$ **then**
                $auxMaxFeatureProspects = \{\} + maxFeatureProspects[j]$
            end if
        end for
        $trainingSetX' \leftarrow dataSetX[auxMaxFeatureProspects]$
        $trainingSetY$
        $testSetX' \leftarrow dataSetX[auxMaxFeatureProspects]$
        $testSetY$
        $model' \leftarrow SVM(trainingSetX, testSetY)$
        if $f1Score(model'.fit(testSetX), testSetY) \geq currMaxScore$ **then**
            $currMaxFeatureProspects \leftarrow auxMaxFeatureProspects$
            $currMaxScore \leftarrow f1Score(model'.fit(testSetX), testSetY)$
        end if
    end for
    $numCols \leftarrow count(currMaxFeatureProspects)$
    $globalMaxScore \leftarrow currMaxScore$
    $maxFeatureProspects \leftarrow currMaxFeatureProspects$
end while
return $maxFeatureProspects$

to continue reducing the feature space. If the space is large enough,the list of feature is shuffle; this step is very crucial to force randomness on the feature selection. For each j_{th} on the current i_{th} , we check on the covering array if the cell i, j its 1 to select the j_{th} feature on the $maxFeatureProspects$ feature and append it into $auxMaxFeatureProspects$.

We can proceed then to create a subset from the global dataset with the selected features and split it into training and test samples. Finally, we get the score of the current model and compare it against the $currMaxScore$; if it turn out to be greater, it will assign $auxMaxFeatureProspects$ to

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad a)$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad b)$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad c)$$

$$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad d)$$

FIGURE 1: **This image shows the reduction of the CA to match length of the selected features. this samples assumes that at each iteration is selecting $N - i$ features and as result the covering array is matching that length**

$currMaxFeatureProspects$.

At the end of the nested loop, $currMaxFeatureProspects$ will be the new new $maxFeatureProspects$ and the $countRow$ will be updated with the new lenght of $maxFeatureProspects$ and the process is repeated again to further reduce the feature space. The step of setting the count row to the length of the length of new selected feature, is very crucial because we want to reduce the covering array with the same length of the selected features, as shown on the Figure 1 .

E. ITERTIVE COVERING ARRAY FEATURE SELECTION

the II-E has shown and documented solid results on classifying cacao nibs, but one thing is failing to do, it's to represent accurately each interaction between the selected N' features on the covering array as always it's being shrunk to match the number of selected features; this indeed violates the condition for covering arrays that each sub array of columns of length t has v^t tuples, because when it is shrunk we cannot be sure that this condition is met at each iteration; that is why in this paper we propose ICAFS, this algorithm is pretty similar to , the only variant , we create a new one from the selected set of features.

The algorithm to create the CA is *IPOG* and it is important to mention that since IPOG is deterministic it produces the same CA, so it's important to shuffle the subset of features selected to give a factor of randomness. The ICAFS is described on the algorithm 2

This algorithm firstly creates a new covering array from the dataset labels' using IPOG algorithm, then for each j on the current row i, within the covering array, it selects the current label if the element on CA is 0; otherwise is ignored. With the selected features it creates the respective training and test to later create a model base on SVM algorithm.

If the score is greater than that the the $currMaxScore$, we re-asgin the new $currMaxScore$ with the score seen so far. At the end we set the $globalMaxScor$ with the

Algorithm 5 ICAFS($dataSetX$, $dataSetY$)

$globalMaxScore \leftarrow 0.0$
$maxFeatureProspects \leftarrow labels(dataSetX)$
$maxNumberOfFeatures \leftarrow count(dataSetX)$
while $i \leq maxIterations$ **do**
  $P \leftarrow constructParamaterSet\_WithBinaryValueEach(maxFeatureProspects)$
  $CA \leftarrow IPOG(t, P)$
  $numRows \leftarrow CA.rows$
  $numCols \leftarrow CA.cols$
  $currMaxFeatureProspects \leftarrow \{\}$
  $currMaxScore \leftarrow 0$
  $shuffle(maxFeatureProspects)$
  for i in $range(0, numRows)$ **do**
    $auxMaxFeatureProspects \leftarrow \{\}$
    for j in $range(0, numCols)$ **do**
      if $CA[i][j] = 1$ **then**
        $auxMaxFeatureProspects = \{\} + maxFeatureProspects[j]$
      end if
    end for
    $trainingSetX' \leftarrow dataSetX[auxMaxFeatureProspects]$
    $trainingSetY$
    $testSetX' \leftarrow dataSetX[auxMaxFeatureProspects]$
    $testSetY$
    $model' \leftarrow SVM(trainingSetX, testSetY)$
    if $f1Score(model'.fit(testSetX), testSetY) \geq currMaxScore$ **then**
      $currMaxFeatureProspects \leftarrow auxMaxFeatureProspects$
      $currMaxScore \leftarrow f1Score(model'.fit(testSetX), testSetY)$
    end if
  end for
  $numCols \leftarrow count(currMaxFeatureProspects)$
  $globalMaxScore \leftarrow currMaxScore$
  $maxFeatureProspects \leftarrow currMaxFeatureProspects$
end while
return $maxFeatureProspects$

$currMaxScore$ and the process is repeated until there is no more feature to reduce or the max number of iteration is reached.

III. METHODOLOGY FOR COMPARISON

The main objective of this document is to compare one solid implementation of a Feature Selection algorithm, BBA, against two covering array feature selection approaches; CAFS and ICAFS. Finally, the results will be presented on the respective section.

To assess the effectiveness of the algorithms presented, this

paper will follow the methodology shown on Fig. 2. Each phase from the methodology it is explained in detail on the following subsections.

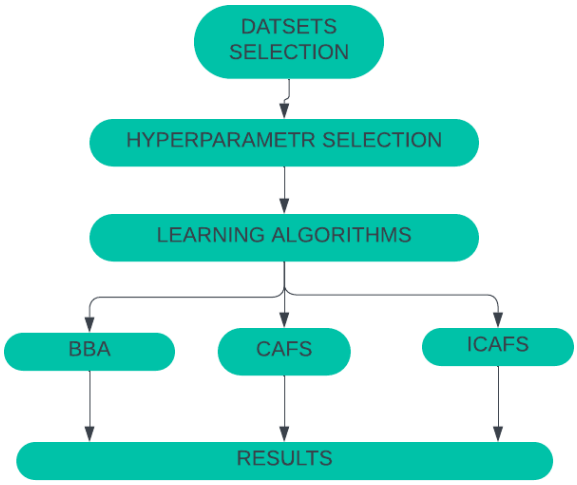


FIGURE 2: Experimental methodology used to compare the three different algorithms.

A. DATASETS SELECTION

The datasets used for the experiments belong to classification tasks. The properties are listed on Table (No olvidar de hacer la tabla).

TABLE 2: Caption Placeholder

Dataset	No. Features	Features	Description
Cacao Nibs	1401	NIR Wavelengths from 1100 to 2500	Describes 6 different Cacao amazonian species through NIR wavelengths
Non-plus/plus Algarrobo	24	R, G, B, Edge Red, NIR, EXG, EXGR, NGRDI, NGBDI, RGRI, GBRI, GBRI, CIVE, VEG, RGBVI, MGRVI, NDVI, RVI, DVI, EVI, REVI, NDRE, RERVI, REDVI	Describes two species of Algarrobo trees
Circle In Square	6	Row 3, Col 3	Row 3, Col 4

The Cacao Nibs was extracted and processed using methodologies explained by Castro et al. [2022]. However, Non-Plus/Plus Algarrobo trees was generated by taking hyperspectral photos of Algarrobo trees that belonged to two main species, Non-Plus/ Plus; each sample was processed with specialized techniques, transforming the images into vegetation indices.

CIS it's a synthetic data set that describes whether a given point is outside or inside the circle with two main features. Moreover, CIS includes 4 extra features to increment the dimensionality of the dataset and add noise to the classification task. The Fig. 3 presents the visual representation of the *inside* and *outside* samples in the 2-dimensional feature space.

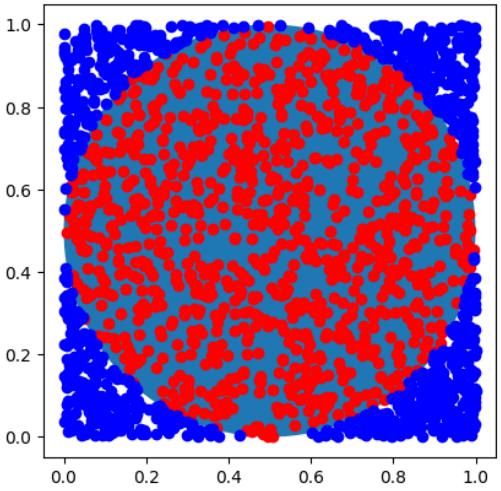


FIGURE 3: **Visual representation of the feature space of the original samples corresponding to the circle in square synthetic dataset**

B. CLASSIFICATION ALGORITHMS

In order to have variability in the results obtained, this document will present two versions of ICAFS and CAFS, using two different learning algorithms.

BBA uses the OPF learning algorithm (*Optimum Path Forest*) (agregar referencia). The OPF algorithm is a graph forest classification algorithm; the available forests group similar samples and each forest represents different classes available in the dataset; hence, during the training phase, each training sample is assigned to the forest with the minimum path cost toward the training sample.

This paper presents two versions of ICAFS and CAFS, one implemented with kNN, and the other with SVM. kNN is an algorithm that works very similarly to OPF; it was chosen in order to have a fair comparison between BBA, CAFS, and ICAFS. On the other hand, a third version of the algorithms was designed with SVC or Support Vector Machine.

SVM or Support Vector Machine is a learning algorithm used for classification problems. The main goal consists of correctly classifying a training sample on the correct side of a plane delimited by a decision boundary (referencia a figura). The training phase consists of minimizing such decision boundary in order to increase the chance of a correct classification.The decision boundary is delimited and supported by vectors from two different classes, hence the name Support Vector.

kNN or k-Nearest Neighbors is a nonparametric learning algorithm; this means that the only hyperparameter to configure is the number of closest neighbors that each training sample will be compared against to assign the correct label. During the training phase, each sample will be compared against the number of closets chosen neighbors, and it will assign the label with major number of frequency to the incoming sample. The distances used are the Euclidean and Coisen distances. One major drawback of kNN is that it has to keep in memory all the training data for the evaluation phase.

C. HYPER PARAMETER SELECTION

The hyperparameters chosen for each of the FS algorithms can be found in the following listing. ICAFS and CAFS will use SVC without specifying any hyperparameter. For kNN, the number of neighbors is set to **3**.

1) **BBA** : The best hyper parameters that showed a good performance regarding the features selected and the accuracy were the following : $f_{min} = 1$,$f_{max} = 15$, $A = 1.0$, $r = .7$, $\alpha = .95$, $\gamma = .6$
2) **CAFS**: Only a CA that matches the number number of existing features for each of the dataset.
3) **ICAFS**: CIS and Non-plus/Plus Algarrobo trees were set with a **strength** $t = 3$ for the creation of CA. Cacao nibs with $t = 2$. The alphabet for each CA is $Z_v = [0, 1]$. Cacao Nibs presents a huge number of features, So a reasonable interaction between every $t = 2$ column was chosen; otherwise the computational time would increase exponentially due to the IPOG complexity addressed by Lei et al. [2007].

IV. RESULTS AND DISCUSSION

A. CAFS RESULTS

Figure 4a presents the evolution of performance as the iCAFS algorithm iterates to reduce the number of features. The graphs describe the number of features selected up to 20 iterations. Initially, the cacao dataset is composed of 1401 features, and the reduction on every iteration reduces to an amount to a half compared to the previous one. The features space gets reduced considerably until the 10_{th} iteration; achieving a selection of 6 features. According to Figure 4a, the behaior of the iCAFS algorithm over the Cacao Nibs dataset, showed a high performance on each iteration by reducing the feature space, whereas the accuracy score increases on each step.

Figure 4 shows how the number of variables in the feature space is reduced from the original 6 features, to the correct 2 dimensions in which the circle was originally drawn. This synthetic data set its made of noisy features, because only the first two columns contains the data points important to describe the problem; so this result not only show that feature space got reduced, but also that with the correct hyperparameters on the creation of the model, it can guide the search to features that are the most relevant, or in other words, that provide the classification information related to the underlying nature of the classification problem. Figure 4 presents a loss of accuracy by selecting a small subset of features, which can be interpreted as a wrong selection strategy, or correlated features may still contain relevant information.

B. ICAFS RESULTS

Fig. 4 presents the results of ICAFS with kNN. In Fig. 4.a showed an increase in the score while considerably reducing the feature space for the cacao nib data set and converged in only a few iterations. However, in Figure 4b shows the the accuracy score for the nonplus / plus data set decreased starting the 6_{th} iteration due to the reduction in the feature space.

On the other hand, the ICAFS SVC did not behave well compared to the kNN version; Just by comparing Figure 4a and Figure 4b, one can notice that SVC underperformed considerably and the same can be said for Non-plus / plus. ICFAS SVC selected 3 features for Circle in the square; one more feature than the kNN version. It is important to note that SVC was used without fine-tuning any hyperparmeter, and this could impact the results obtained; therefore, an additional study focusing on choosing the best hyperparameters should be considered for future work.

Table 3 shows the best accuracy scores found with the best number of features selected for ICAFS kNN and ICAFS SVC.

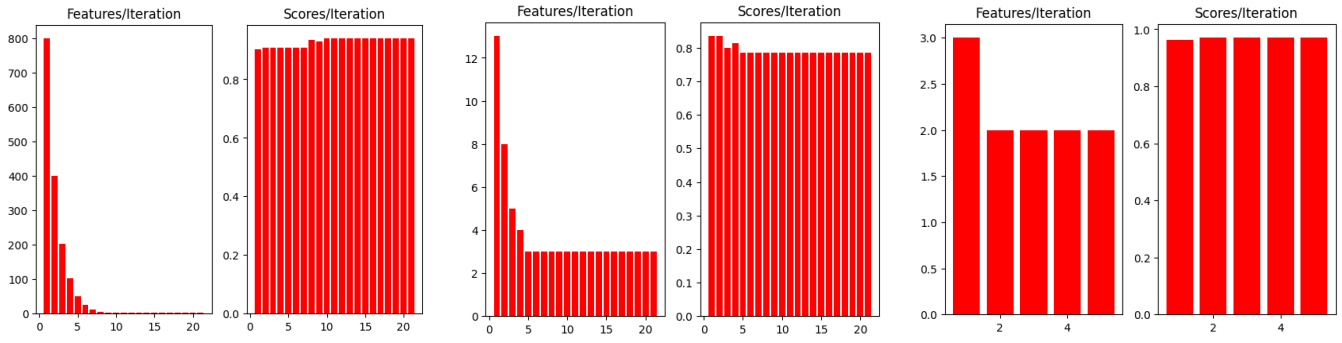
C. BBA RESULTS

The last three rows of Table 3 present the results of the BBA algorithm after three iterations. This algorithm was able to select three features out of 1401 in the cacao dataset, with 82.98 percent of accuracy. It is the same case for nonplus/plus Algarrobo that selected 4 features, and it was not able to find the two most relevant features in the circle in square dataset.

V. CONCLUSION

In this paper, a new iterative covering arrays feature selection algorithm was proposed for the reduction of feature dimensionality in classification problems. In experiments, the proposed iCAFS algorithm overcame the other approaches used for comparison, including the CAFS algorithm that employs covering arrays as its underlying search strategy. According to the results, the iCAFS algorithm assembled with the k-NN classifier was the best approach, although this classifier is the simplest, and the slowest in classification stage.

Future research directions may include the use of recent deep techniques for classification, and the incorporation of an embedded covering array guided search of hyperparameters of neural classifiers. The application of the algorithm in other chemometric problems is also an issue to be addressed. Similarly, other algorithms to generate covering arrays may provide a trade-off between complexity, and a more accurate exploration of the feature space in terms of the interaction between variables.



a) *b)* *c)*

FIGURE 4: **The number of features selected, and the maximum score reached during each iteration for each dataset, using ICAFS kNN.** *a) Cacao nibs*; *b) Non-Plus/Plus Algarrobo*; *c) CIS* **remained stable, selecting only 2 features**

TABLE 3: **Comparison across the different dataset processed with ICAFS kNN and ICAFS SVC**

	Dataset	*Accuracy (%)*	*Number of Features*	*Features selected*	*Max No. Iterations*
ICAFS kNN	Cacao Nibs	**93.907**	3	1736, 1993, 2089	10
	Non-Plus/Plus Algarrobo	**81.443**	4	R, EXGR, MGRVI, EVI	4
	Circle In Square	**97.249**	2	X,Y	3
ICAFS SVC	Cacao Nibs	82.986	3	1734, 1926, 2310	10
	Non-Plus/Plus Algarrobo	79.00	4	Edge Red, GBRI.1, NDVI, REDVI	4
	Circle In Square	96.153	3	X,Y,D	3
CAFS kNN	Cacao Nibs	80.152	3	1734, 1926, 2310	10
	Non-Plus/Plus Algarrobo	74.997	4	Edge Red, GBRI.1, NDVI, REDVI	4
	Circle In Square	90.786	3	X,Y,D	3
CAFS SVC	Cacao Nibs	81.147	3	1734, 1926, 2310	10
	Non-Plus/Plus Algarrobo	80.323	4	Edge Red, GBRI.1, NDVI, REDVI	4
	Circle In Square	92.428	3	X,Y,D	3
BBA	Cacao Nibs	81.961	3	1734, 1926, 2310	10
	Non-Plus/Plus Algarrobo	73.103	4	Edge Red, GBRI.1, NDVI, REDVI	4
	Circle In Square	94.432	3	X,Y,D	3

REFERENCES

Castro, W., De-la Torre, M., Avila-George, H., Torres-Jimenez, J., Guivin, A., and Acevedo-Juárez, B. (2022). Amazonian cacao-clone nibs discrimination using nir spectroscopy coupled to naïve bayes classifier and a new waveband selection approach. Spectrochimica Acta Part A: Molecular and Biomolecular Spectroscopy, 270:120815.

Guha, R., Chatterjee, B., Khalid Hassan, S. K., Ahmed, S., Bhattacharyya, T., and Sarkar, R. (2021). Py_FS: A Python Package for Feature Selection Using Meta-Heuristic Optimization Algorithms, pages 495–504. Springer Singapore.

Guyon, I. and Elisseeff, A. (2003). An introduction to variable and feature selection. J. Mach. Learn. Res.,

3(null):1157–1182.

Izquierdo-Marquez, I., Torres-Jimenez, J., Acevedo-Juárez, B., and Avila-George, H. (2018). A greedy-metaheuristic 3-stage approach to construct covering arrays. Information Sciences, 460–461:172–189.

Lei, Y., Kacker, R., Kuhn, D. R., Okun, V., and Lawrence, J. (2007). Ipog: A general strategy for t-way software testing. In 14th Annual IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS'07). IEEE.

Lei, Y. and Tai, K. (1998). In-parameter-order: a test generation strategy for pairwise testing. In Proceedings Third IEEE International High-Assurance Systems Engineering Symposium (Cat. No.98EX231), pages 254–261. IEEE Comput. Soc.

Liu, H. and Motoda, H., editors (2007). Computational Methods of Feature Selection. Chapman and Hall/CRC.

Nakamura, R. Y. M., Pereira, L. A. M., Rodrigues, D., Costa, K. A. P., Papa, J. P., and Yang, X.-S. (2013). Binary Bat Algorithm for Feature Selection, pages 225–237. Elsevier.

Torres-Jimenez, J., Izquierdo-Marquez, I., and Avila-George, H. (2019). Methods to construct uniform covering arrays. IEEE Access, 7:42774–42797.

Wang, S., Tang, J., and Liu, H. (2016). Feature Selection, pages 1–9. Springer US.

Yang, X. S. (2011). Bat algorithm for multi-objective optimisation. International Journal of Bio-Inspired Computation, 3(5):267.